MSC THESIS · GRADED 10/10

# Fraud Detection, End to End

Machine Learning for Fraud Detection in Online Financial Transactions

**3.5%**
fraud rate in the data

**590K+**
labelled transactions

**0.919**
ROC AUC, best model

Konstantinos Koutsompinas · National and Kapodistrian University of Athens

### Severe Class Imbalance

Fraudulent transactions represent only ~3.5% of all activity. Models can appear accurate while still failing to detect fraud reliably.

### Evolving Fraud Patterns

Fraud strategies change over time, so fixed rules and manual review are limited when patterns become more complex.

### False Positives vs. False Negatives

Catching more fraud usually means flagging more legitimate transactions. The operating threshold must balance fraud losses against analyst workload and customer friction.

### Operational Constraints

Fraud detection systems must support timely decisions, minimize customer disruption, and remain practical for investigation workflows.

### Interpretability vs. Accuracy

Ensemble models can perform strongly, but their complexity makes explainability important for understanding model decisions.

**3.5%**
fraud rate
in IEEE-CIS

**590K+** Transactions

**434** Features

**3** ML Models

**4** RQs

## RQ 1 Model Discrimination

How effectively do gradient boosting models (XGBoost, LightGBM, CatBoost) discriminate fraudulent from legitimate transactions under ROC-AUC evaluation?

## RQ 2 Impact of Downsampling

How does majority-class downsampling (1:5 ratio) affect model performance compared to training on the original imbalanced data distribution?

## RQ 3 Feature Set Reduction

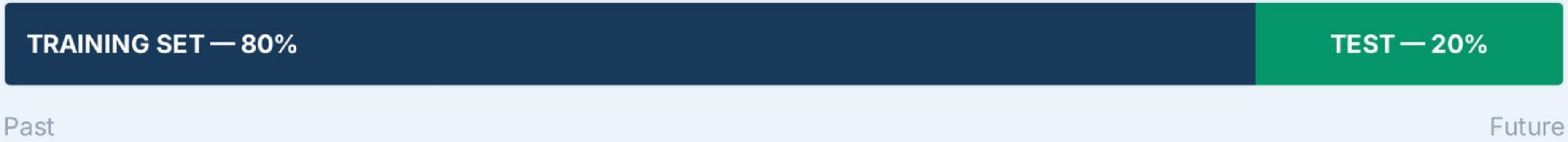
Can a reduced feature set, selected through cross-model feature importance agreement, preserve the discriminatory power of the full feature space?

## RQ 4 Threshold Tuning Effects

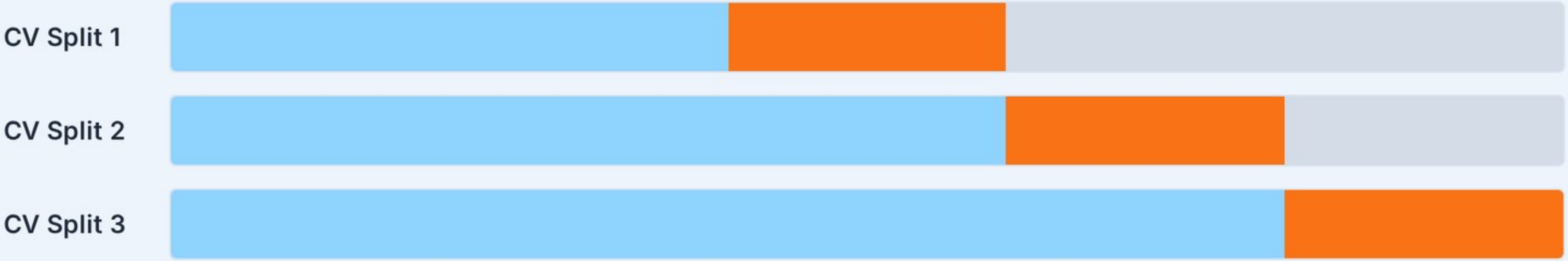
How does altering the decision threshold change fraud detection behavior — specifically the recall–precision trade-off relevant to operational deployment?

A further objective is to assess the added value of engineered aggregate behavioural features such as `_mean`, `_rel`, `_avg`, `_std`, and `_freq`, which capture historical and user-level patterns beyond raw transaction fields.

→ Chronological order (`TransactionDT`) — no shuffling and no standalone validation split




**Time-Series Cross-Validation (within training set)**




Chronological evaluation is critical in fraud detection — random splitting would leak future patterns into training.

### No Data Leakage

Future information never used to train on past data

### 80/20 Holdout

First 80% used for training, final 20% reserved as unseen test data

### 3-Fold CV

Validation performed inside the training set with expanding time-series folds

# METHODOLOGY PIPELINE

1

### EDA

Analyze class imbalance, missingness, temporal structure, and fraud-related behavioral patterns

2

### Preprocessing

Remove extreme-missingness fields and prepare the time-aware dataset for modeling

3

### Feature Engineering

Create aggregate behavioral features such as mean, relative, avg, std, and frequency signals

4

### Initial Training

Train XGBoost, LightGBM, and CatBoost with Optuna tuning and time-series cross-validation

5

### Downsampling

Test majority-class downsampling to improve learning under severe class imbalance

6

### Feature Reduction

Use cross-model SHAP importance agreement to retain the most informative compact feature set

7

### Threshold Tuning

Adjust the classification threshold to study the fraud-capture versus false-positive trade-off

Time-series cross-validation was used throughout to prevent data leakage and simulate real-world deployment conditions.

## SHAP Feature Importance: Selection Rationale

**Table 4: Mean |SHAP| Importance Across XGBoost, LightGBM & CatBoost**

| | | |
|---|---|---|
| 1 | C1 | 0.422 |
| 2 | C13 | 0.239 |
| 3 | V70 | 0.180 |
| 4 | uid_V258_mean | 0.143 |
| 5 | ProductCD_TransactionAmt_rel | 0.141 |
| 6 | uid_C1_mean | 0.136 |
| 7 | DeviceInfo | 0.132 |
| 8 | V258 | 0.131 |
| 9 | uid_V283_mean | 0.130 |
| 10 | uid_C13_std | 0.130 |
| 11 | uid_C13_rel | 0.129 |
| 12 | card1_id_02_mean | 0.114 |
| 13 | P_emaildomain | 0.107 |
| 14 | uid_C13_mean | 0.104 |
| 15 | uid_D2_std | 0.101 |

### Global Selection

Mean absolute SHAP ranks the variables driving model predictions across the trained gradient boosting models.

### Cross-Model Agreement

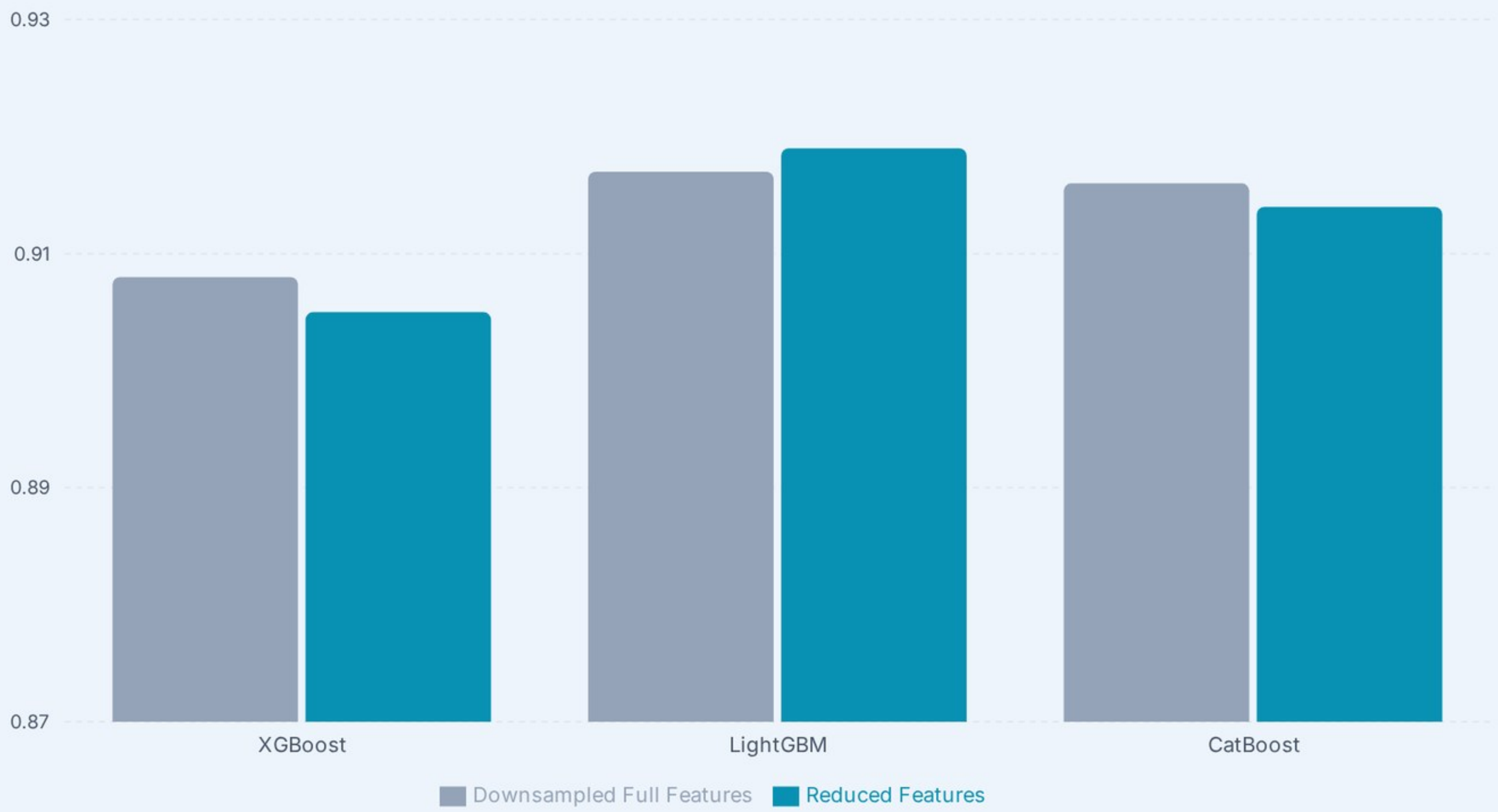
Features in the top 30% for at least two models were kept, reducing 748 model inputs to 215.

### Local Explanations

The same SHAP machinery supports per-transaction attribution in the live demo.

**These SHAP rankings justify the reduced-feature experiment: engineered behavioral aggregates appear alongside original C, V and identity/device fields.**

**Compact models remain highly competitive — LightGBM slightly improves to 0.919 ROC-AUC with reduced features**




### Performance Maintained

All three models retain near-identical ROC-AUC after compression. LightGBM marginally improves to 0.919.

### Top Engineered Features

Behavioral aggregates (_mean, _rel, _avg, _std, _freq) consistently rank as the strongest predictors.

### SHAP-Driven Selection

Top 30% importance per model, kept when ≥2 models agree — reduces model inputs from 748 to 215.

### Deployment Advantage

Fewer features = faster inference, simpler pipelines, lower maintenance cost in production.

**Lowering the reduced-feature model threshold to 0.1 sharply increases fraud recall at the cost of precision**

## XGBoost

**Threshold 0.5**
Recall: 0.678
Precision: 0.300

↓

threshold → 0.1

**Threshold 0.1**
**Recall: 0.922**
**Precision: 0.085**

## LightGBM

**Threshold 0.5**
Recall: 0.661
Precision: 0.348

↓

threshold → 0.1

**Threshold 0.1**
**Recall: 0.895**
**Precision: 0.110**

## CatBoost

**Threshold 0.5**
Recall: 0.720
Precision: 0.298

↓

threshold → 0.1

**Threshold 0.1**
**Recall: 0.938**
**Precision: 0.076**

The thesis frames the threshold as an operational lever tied to fraud-loss tolerance and available review capacity.

CHAMPION SCORECARD

# ROC-AUC Across All Configurations

| Model | Baseline (Imbalanced) | Config 2 (Downsampled) | Config 3 (Reduced Features) |
|---|---|---|---|
| LightGBM | 0.918 | 0.917 | 0.919 |
| CatBoost | 0.910 | 0.916 | 0.914 |
| XGBoost | 0.896 | 0.908 | 0.905 |

LightGBM provides the most stable ROC-AUC performance. Thesis rationale: histogram-based split finding handles the high-dimensional feature space efficiently and may regularize under severe imbalance.

# Live Demo

LightGBM · Fraud Probability + Explainability

ROC-AUC: **0.919** PR-AUC: **0.566**

**PIPELINE**

- ✓ Raw Data
- ✓ Preprocessing
- ✓ Feature Eng.
- ✓ Selection
- ✓ Model
- ✓ SHAP

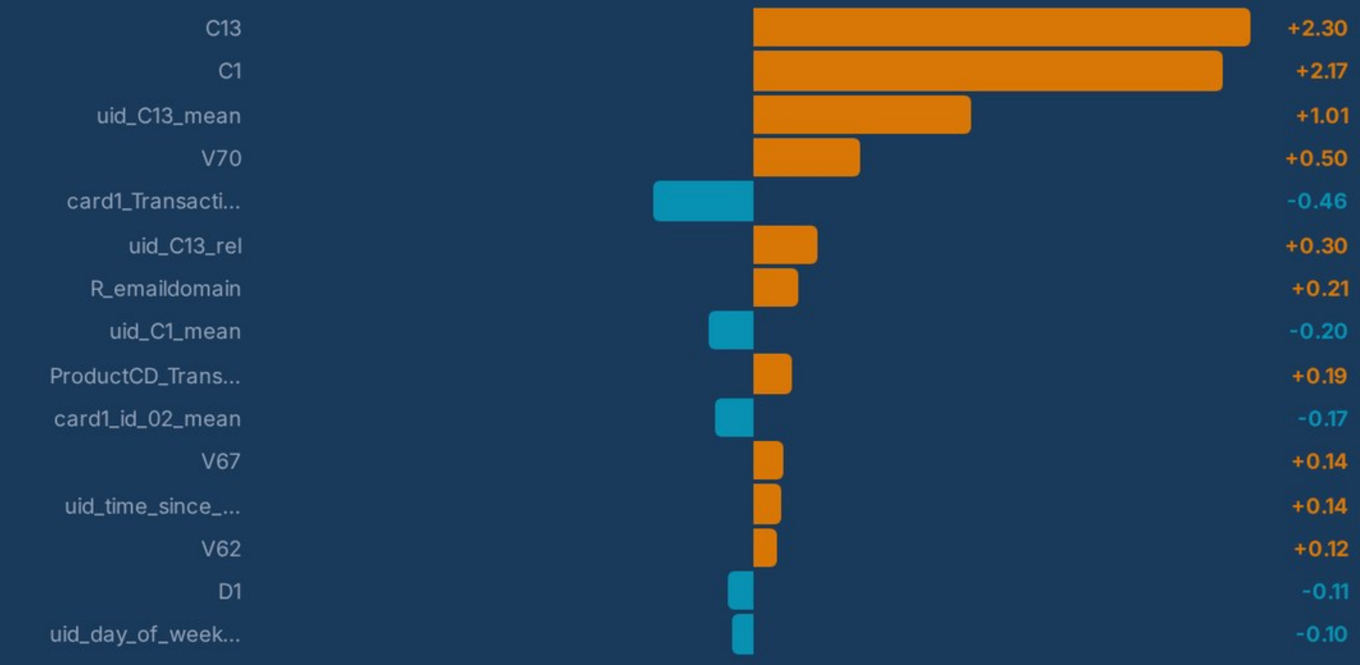
**SELECT A TRANSACTION**

⚠ **High Risk** | ✓ **Low Risk** | ~ **Mid Risk**


98% fraud probability

Raw model score from 0% to 100%

**What drove this score?**




SHAP values are log-odds contributions. Positive values increase fraud probability.

Try Another Transaction →

## KEY CONCLUSIONS

1. Gradient boosting models are highly effective for fraud detection on the IEEE-CIS dataset, achieving strong discrimination without deep learning complexity.
2. LightGBM was the most dependable model overall, with ROC-AUC ≈ 0.918–0.919 across configurations — making it the recommended baseline choice.
3. Engineered aggregate behavioral features ranked among the most important predictors and helped preserve performance after feature reduction.
4. Threshold tuning substantially increased recall but reduced precision, showing that deployment requires explicit trade-off management.

## FUTURE RESEARCH

### Cost-Aware Thresholds

Select thresholds using review capacity or explicit false-positive and false-negative costs

### Probability Calibration

Improve whether predicted risk scores reflect true fraud likelihood

### Richer Features & Data

Use behavioral histories, temporal patterns, relational signals, and larger transaction datasets

### Robustness Extensions

Explore ensemble strategies and drift-aware evaluation as fraud patterns evolve

# Graded 10/10

MSc thesis, National and Kapodistrian University of Athens

Results, code and the full write up

**github.com/koutsompinask/MSC-thesis**

**Konstantinos Koutsompinas**

Full Stack Software Engineer